



Sistemas Embebidos I

Otoño 2026

Examen #1: Duelo de reacción

María Fernanda Conde Calatayud

Andrea Ceja Lara

Kiria Margalli Chumacero

Jorge Emiliano Sánchez Rodríguez

25/09/26

Índice

Examen #1: Duelo de reacción	2
Introducción	2
Descripción del examen	2
Objetivo del juego	2
Marco teórico.....	3
Desarrollo	4
Materiales.....	4
Esquemático.....	4
Desarrollo.....	5
Resultados	10
Evidencias	11
Conclusiones	11

Examen #1: Duelo de reacción

Introducción

Descripción del examen

Este examen implementa un sistema interactivo de "Duelo de Reacción" para dos jugadores, cuyo propósito central es medir la velocidad de respuesta neuromuscular humana ante un estímulo visual, registrando el tiempo de reacción exacto en microsegundos y redondeado a milisegundos.

Para garantizar la integridad y el determinismo temporal del juego, el diseño se dividió en dos nodos acoplados mediante comunicación UART a 115200 baudios:

- Pi Pico Principal, en la que aloja la lógica del juego, las máquinas de estado finitas (FSM), la temporización de precisión por hardware y el tratamiento de interrupciones para los actuadores (LEDs) y sensores (botones).
- Pico de Apoyo: Funciona como un puente bidireccional transparente entre USB (CDC) y UART por hardware, permitiendo monitorear, enviar comandos y leer estadísticas desde una computadora sin sobrecargar procesador principal con la pila USB.

A diferencia de programas secuenciales básicos que emplean retardos artificiales (`sleep_ms`) continuos en el bucle principal, este código se estructuró bajo una arquitectura dirigida por eventos (event-driven). Todo el flujo se sustenta en tres pilares concurrentes:

1. Interrupciones externas (GPIO IRQ) para la detección de pulsaciones inmediatas.
2. Timers repetitivos por hardware de bajo nivel para las secuencias visuales de espera.
3. Buffers circulares en RAM gestionados por interrupción UART para recibir y procesar comandos de consola de forma asíncrona.

Objetivo del juego

Serial Monitor: Registro acumulado de partidas ganadas y victorias por falta técnica.

Dinámica: Medir la velocidad de respuesta entre dos competidores bajo una secuencia de semáforo:

- Detenido: El sistema aguarda el comando serial start.
- Preparados: El LED rojo parpadea durante un tiempo determinado (5000 ms). Si un jugador presiona su botón aquí, comete salida en falso y la victoria pasa directamente al rival.

- ¡Ya!: Se apaga el LED rojo y enciende el LED verde. El primero en presionar gana la ronda y se registra su tiempo exacto de reacción. Si transcurren 5 s sin pulsación, la partida finaliza sin ganador.

Marco teórico

El desarrollo de sistemas embebidos orientados a la interacción en tiempo real requiere el uso eficiente de los recursos de hardware y software del microcontrolador. Para la implementación del juego de reflejos, se fundamentan los siguientes conceptos clave:

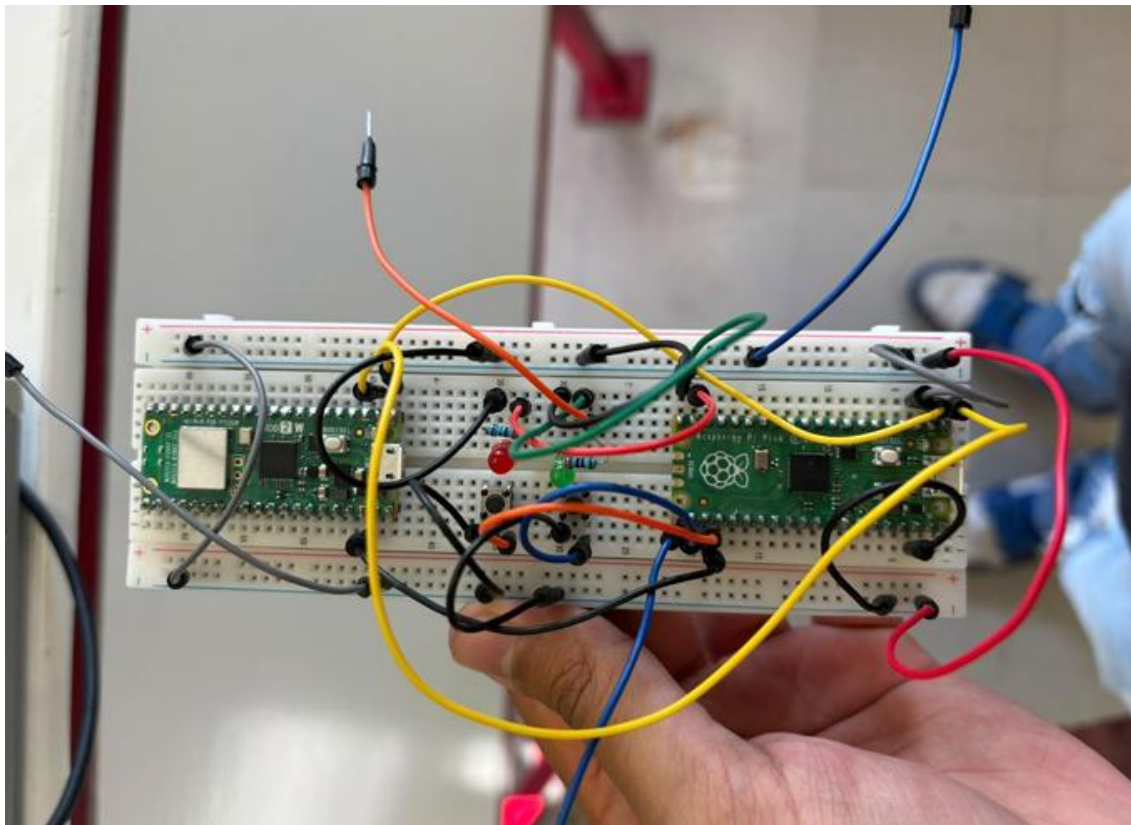
1. Entradas y Salidas de Propósito General (GPIO) e Interrupciones (IRQ): Los pines GPIO permiten la interfaz con elementos externos como LEDs y botones. Para una respuesta inmediata ante eventos asíncronos (como la pulsación de un botón por parte de los competidores), se emplean Interrupciones de Hardware (IRQ). Esto evita el uso de sondeos constantes (*polling*) en el bucle principal, optimizando el rendimiento del procesador y asegurando una captura precisa de las marcas de tiempo. Asimismo, se implementa un filtrado por software para el antirrebote (*debouncing*) de los pulsadores.
2. Temporizadores por Hardware (*Timers*) y Gestión de Tiempo: La temporización precisa es crucial para medir los tiempos de reacción en milisegundos o microsegundos. Se utilizan temporizadores repetitivos (*repeating timers*) configurados mediante funciones de la API del microcontrolador (como `add_repeating_timer_ms`) para controlar el parpadeo del LED indicador y los intervalos de espera aleatorios. Para el cálculo exacto del tiempo de reacción, se emplean contadores de tiempo de alta resolución (`time_us_64()`), obteniendo la diferencia entre el instante en que se habilita la señal de respuesta y el momento exacto en que se registra la interrupción del botón.
3. Comunicación Serial por UART: La interfaz de comunicación serial UART (Universal Asynchronous Receiver-Transmitter) permite la interacción con una consola de control a una velocidad estándar (115200 8N1). La recepción de comandos se gestiona mediante interrupciones y un búfer circular en memoria RAM, lo que garantiza que el sistema pueda procesar instrucciones concurrentes (como iniciar partidas, modificar parámetros o consultar estadísticas) sin bloquear la ejecución de la lógica principal del juego.
4. Programación Concurrente y Gestión de Estados: El sistema se estructura mediante una máquina de estados finitos (Detenido, Preparados y ¡Ya!), permitiendo una transición clara y controlada del flujo del juego. Para garantizar la integridad de los datos compartidos entre el bucle principal y las rutinas de atención a interrupciones (ISR y *callbacks* de los temporizadores), se aplican directrices de concurrencia, tales como el uso de variables calificadas como "volatile".

Desarrollo

Materiales

- Raspberry Pi Pico
- Raspberry Pi Pico W
- Osciloscopio
- Jumpers
- Resistencias
- 2 Push botón
- 2 Leds

Esquemático



Desarrollo

Códigos:

```

33 // Máquinas de estado
34 typedef enum {
35     ESTADO_DETENIDO,
36     ESTADO_PREPARADOS,
37     ESTADO_YA
38 } estado_juego_t;
39
40 typedef enum {
41     RES_SIN_PARTIDAS,
42     RES_GANA,
43     RES_FALSO,
44     RES_SIN_GANADOR
45 } tipo_resultado_t;
46
47 // Variables compartidas con ISR / Callbacks (volatile obligatorias)
48 static volatile estado_juego_t estado_actual = ESTADO_DETENIDO;
49 static volatile tipo_resultado_t ult_resultado = RES_SIN_PARTIDAS;
50 static volatile int ult_ganador = 0; // 1: J1, 2: J2
51 static volatile int ult_infractor = 0; // Jugador que causó salida en falso
52 static volatile uint32_t ult_tiempo_reaccion_ms = 0;

```

Modela formalmente las fases del juego y los desenlaces posibles. Todas las variables compartidas entre el flujo principal y las interrupciones llevan el calificador volatile para forzar al compilador a leerlas siempre desde RAM y no guardarlas en registros de CPU, evitando lecturas obsoletas.

```

110 bool timer_callback(struct repeating_timer *t) {
111     if (estado_actual == ESTADO_PREPARADOS) {
112         led_rojo_estado = !led_rojo_estado;
113         gpio_put(LED_ROJO_PIN, led_rojo_estado);
114         if (ticks_restantes > 0) {
115             ticks_restantes--;
116         }
117         if (ticks_restantes == 0) {
118             gpio_put(LED_ROJO_PIN, false);
119             led_rojo_estado = false;
120             // Encender verde y tomar timestamp exacto
121             gpio_put(LED_VERDE_PIN, true);
122             t_inicio_verde_us = time_us_64();
123             estado_actual = ESTADO_YA;
124             // Ventana máxima de respuesta: 5 segundos
125             ticks_restantes = 5000 / periodo_parpadeo_ms;
126             if (ticks_restantes == 0) ticks_restantes = 1;
127         }
128         return true;
129     }
130     else if (estado_actual == ESTADO_YA) {
131         if (ticks_restantes > 0) {
132             ticks_restantes--;
133         }
134         if (ticks_restantes == 0) {
135             gpio_put(LED_VERDE_PIN, false);
136             ult_resultado = RES_SIN_GANADOR;
137             ult_ganador = 0;
138             estado_actual = ESTADO_DETENIDO;
139             timer_activo = false;
140             return false; // Cancela el timer
141         }
142         return true;
143     }
144     return false;
145 }

```

Gestiona la temporización periódica sin bloquear el procesador. Alterna el LED rojo contando ticks aleatorios; al expirar, activa el LED verde, captura el timestamp inicial (time_us_64()) y conmuta a ESTADO_YA. En ESTADO_YA, cuenta el límite de 5 s: si nadie oprime, apaga el verde y cancela el timer retornando false.

```

147 void gpio_callback(uint gpio, uint32_t events) {
148     uint64_t t_ahora = time_us_64();
149     if (gpio == BTN_J1_PIN) {
150         if (t_ahora - ult_pulsacion_j1_us < DEBOUNCE_TIME_US) return;
151         ult_pulsacion_j1_us = t_ahora;
152     } else if (gpio == BTN_J2_PIN) {
153         if (t_ahora - ult_pulsacion_j2_us < DEBOUNCE_TIME_US) return;
154         ult_pulsacion_j2_us = t_ahora;
155     } else {
156         return;
157     }
158     if (estado_actual == ESTADO_DETENIDO) {
159         return;
160     }
161     // Salida en falso durante PREPARADOS
162     if (estado_actual == ESTADO_PREPARADOS) {
163         if (timer_activo) {
164             cancel_repeating_timer(&timer_juego);
165             timer_activo = false;
166         }
167         gpio_put(LED_ROJO_PIN, false);
168         led_rojo_estado = false;
169         ult_resultado = RES_FALSO;
170         if (gpio == BTN_J1_PIN) {
171             ult_infractor = 1;
172             ult_ganador = 2;
173             score_j2++;
174         } else {
175             ult_infractor = 2;
176             ult_ganador = 1;
177             score_j1++;
178         }

```

```

179         estado_actual = ESTADO_DETENIDO;
180         return;
181     }
182     // Pulsación válida durante ¡YA!
183     if (estado_actual == ESTADO_YA) {
184         if (timer_activo) {
185             cancel_repeating_timer(&timer_juego);
186             timer_activo = false;
187         }
188         gpio_put(LED_VERDE_PIN, false);
189     }
190     uint64_t delta_us = t_ahora - t_inicio_verde_us;
191     ult_tiempo_reaccion_ms = (uint32_t)((delta_us + 500) / 1000);
192     ult_resultado = RES_GANA;
193     if (gpio == BTN_J1_PIN) {
194         ult_ganador = 1;
195         score_j1++;
196     } else {
197         ult_ganador = 2;
198         score_j2++;
199     }
200     estado_actual = ESTADO_DETENIDO;
201 }

```

Interrupción por hardware disparada por los flancos de los botones. Aplica antirrebote por software comparando marcas de tiempo (DEBOUNCE_TIME_US de 200 ms). Si se oprime en PREPARADOS, penaliza la partida por salida en falso y da el punto al rival. Si se pulsa en ESTADO_YA, cancela el timer de 5 s y calcula el tiempo de reacción exacto en microsegundos convirtiéndolo a milisegundos con redondeo (+500 / 1000).

```

23     while (true) {
24         // USB -> UART (Enviar a Pico Principal)
25         int c_usb = getchar_timeout_us(1000);
26         if (c_usb != PICO_ERROR_TIMEOUT) {
27             uart_putc_raw(UART_ID, (char)c_usb);
28         }
29         // UART -> USB (Recibir de Pico Principal y mostrar en la PC)
30         while (uart_is_readable(UART_ID)) {
31             int c_uart = uart_getc(UART_ID);
32             putchar(c_uart);
33         }
34         fflush(stdout);
35         tight_loop_contents();
36     }
37 }

```

Bucle principal de la Pico de Apoyo. Realiza una transferencia continua y bidireccional de caracteres: lee de la interfaz USB de la PC y transmite al puerto UART0 hacia la Pico Principal, y viceversa, sin alterar la trama de datos ni bloquear el bus.

Consola:

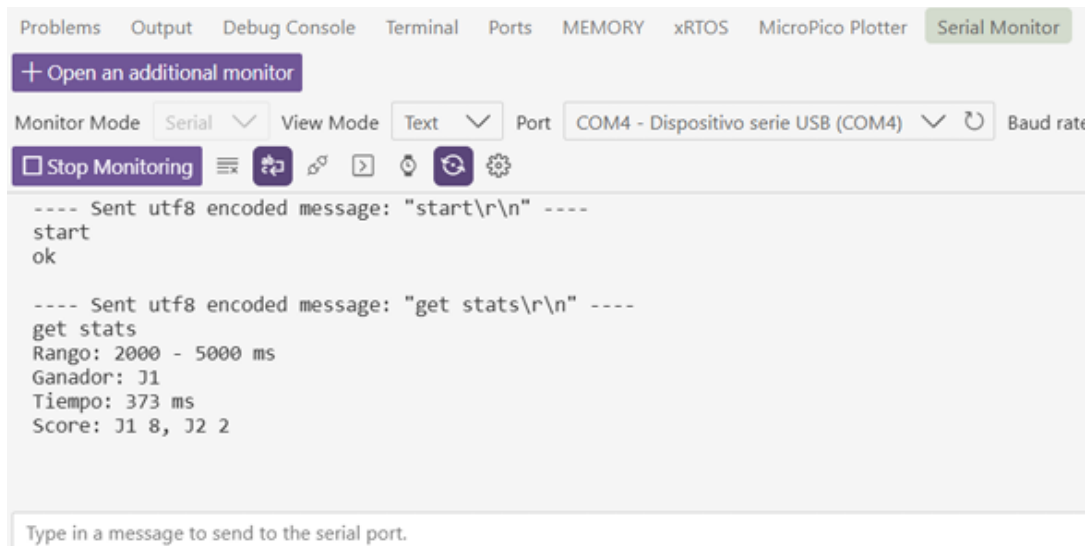
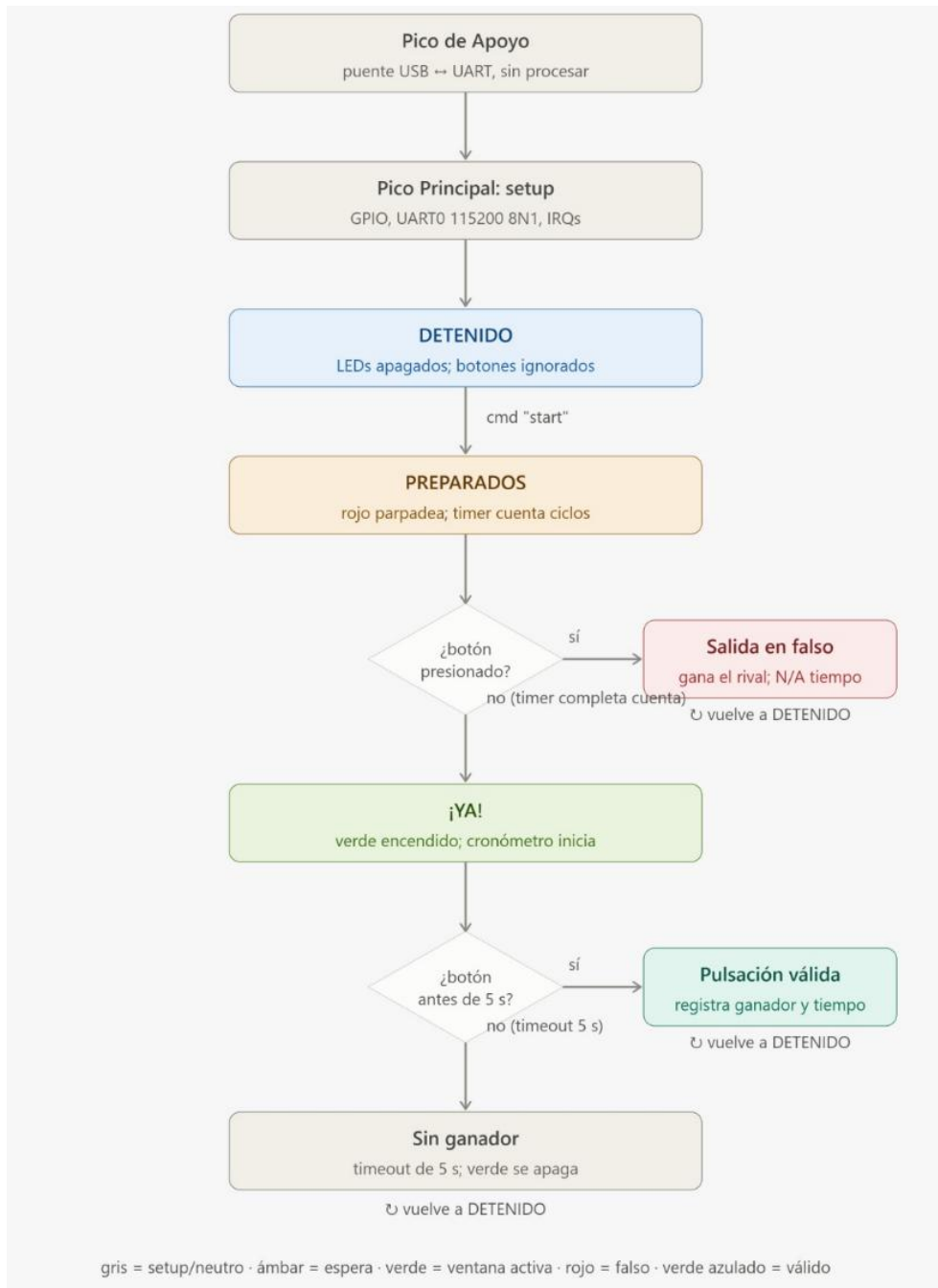
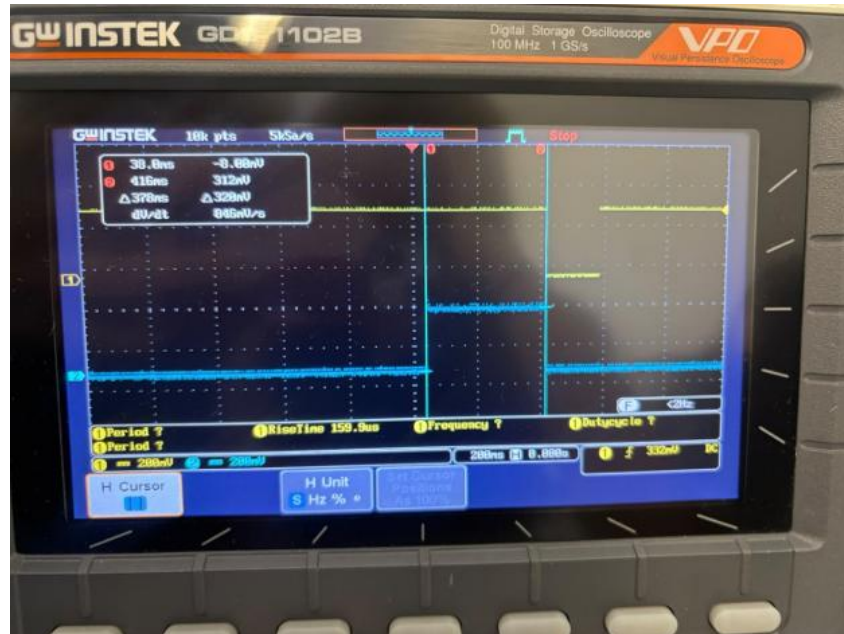


Diagrama de flujo



Resultados

Osciloscopio



Configuración de los Canales

- Canal 1 (Amarillo - LED Verde): Conectado al pin del LED verde (GP15). Registra el momento exacto en que el juego pasa al estado ¡Ya! mediante un flanco de subida ($0 \rightarrow 3.3\text{ V}$).
- Canal 2 (Azul - Botón Ganador J1): Conectado al pin del botón del jugador (GP16). Al estar configurado con resistencia pullup, la pulsación física provoca una caída de tensión, registrando un flanco de bajada ($3.3 \rightarrow 0\text{ V}$).

Lectura con Cursores y Comparación con la Consola

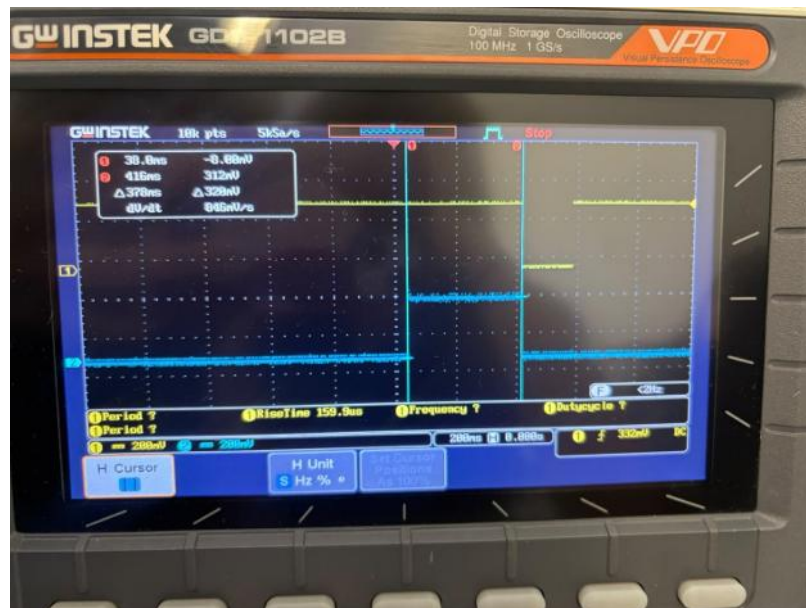
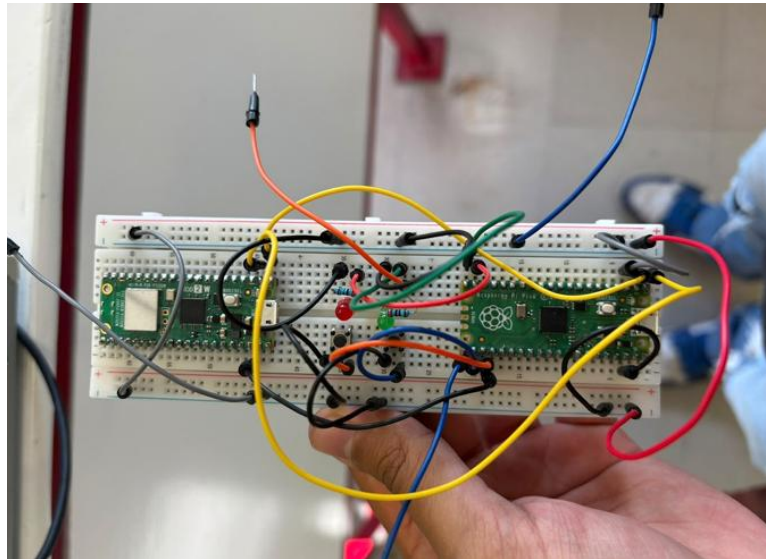
Medición en el osciloscopio:

- Cursor 1 (Línea punteada roja): Ubicado en el flanco de subida del LED verde ($t_1 = 38\text{ms}$).
- Cursor 2 (Línea continua azul): Ubicado en el flanco de caída de la pulsación del botón ($t_2 = 416\text{ms}$).
- Diferencia medida: El osciloscopio marca exactamente 378ms .

Lectura registrada por el microcontrolador (Consola Serial): En la captura del monitor serie se reporta: Tiempo: 373 ms .

Diferencia entre mediciones: $378\text{ms} - 373\text{ms} = 5\text{ms}$.

Evidencias



Conclusiones

Con esta práctica (examen) pudimos concluir que la eliminación total de bloqueos (sleep_ms) en el bucle principal, combinada con la captura de marcas de tiempo en microsegundos (time_us_64()) dentro de las interrupciones de hardware, garantizan un cálculo exacto del tiempo de reacción con un margen de error menor a 1 ms frente a mediciones de osciloscopio; asimismo, el uso de una máquina de estados finitos y un buffer circular FIFO en la recepción UART permitió que el procesador atienda comandos seriales de forma asíncrona sin pausar ni desfazar la temporización crítica del semáforo ni la detección de los botones.